

MiniMax M3 MSA 小 Batch Decode —— B300 实验报告

1. 问题与最终结论

研究问题： 在 NVIDIA B300 (SM100) 上，MiniMax M3 MSA 的小 batch decode (batch ≤ 16) 是否值得从现有 Triton split-K 实现迁移到专用 kernel？ **最终结论：不值得。** 在 B300、TP1、batch≤16 下，Triton split-K baseline 的配置空间已被上游启发式充分探索。配置 sweep 最佳改善仅 1.7%，低于 5% 噪声底线。主要瓶颈是 host launch overhead (eager 模式占 75%) 和 FP8 反量化开销 (比 BF16 慢 21~60%)。即使完全消除 merge kernel，理论加速上限也只有 1.59× (batch=1, chunks=16)。建议使用 CUDA Graph + 默认配置，不新增 CUDA/CUTLASS 路径。

2. 实验环境

项目	配置
GPU	NVIDIA B300 SXM6 AC
Driver	580.126.09
PyTorch	2.14.0+cu130
Triton	3.8.0
NumPy	2.5.3
Python	3.12.3
测量方法	CUDA Event + CUDA Graph replay, warmup 50 次, iterations 200 次

3. 正确性验收

参照实现：FP32 gathered-KV attention ([reference_decode](#))。
容差：BF16 `rtol=1e-2, atol=1e-2`；FP8 `rtol=2e-2, atol=2e-2`。

B300 实测结果

batch	dtype	max_abs_error	cosine	判定
1	BF16	2.44e-4	0.99999535	PASS
1	FP8	2.44e-4	0.99999529	PASS
4	BF16	4.88e-4	0.99999541	PASS
4	FP8	4.88e-4	0.99999547	PASS

4. 性能数据

4.1 Triton FP8 延迟 (μs)

batch	chunks	eager median	eager p95	graph median	graph p95	launch gap
1	16	43.86	49.95	10.88	11.36	32.98
4	16	42.93	49.15	12.98	13.28	29.95
8	8	43.90	51.14	18.24	19.04	25.66
16	4	48.62	55.81	27.23	28.06	21.39
32	2	68.03	69.98	47.65	47.97	20.38

4.2 Triton BF16 延迟 (μs)

batch	chunks	eager median	eager p95	graph median	graph p95
1	16	43.81	50.11	8.96	10.14
8	8	43.42	50.21	12.96	13.28
16	4	44.02	52.03	16.96	17.38

4.3 FP8 vs BF16 对比 (graph 模式)

batch	FP8 (μs)	BF16 (μs)	FP8 超出
1	10.88	8.96	+21%
8	18.24	12.96	+41%
16	27.23	16.96	+61%

4.4 Split-K 时间分解 (μs, eager 模式)

batch	chunks	partial	merge	total	merge%
1	1	76.06	6.14	82.21	7.5%
1	2	51.26	6.14	57.44	10.7%
1	4	38.37	8.70	46.96	18.5%
1	8	31.86	14.88	46.62	31.9%
1	16	29.28	17.52	46.86	37.4%
4	1	76.62	6.14	82.78	7.4%
4	2	51.20	6.14	57.41	10.7%
4	4	38.94	8.56	47.31	18.1%
4	8	32.34	14.74	46.88	31.4%
4	16	31.78	15.23	46.94	32.4%

batch	chunks	partial	merge	total	merge%
8	1	76.70	6.14	82.85	7.4%
8	2	51.42	6.14	57.60	10.7%
8	4	39.22	7.07	46.51	15.2%
8	8	36.10	10.86	46.93	23.2%
8	16	37.73	8.61	46.38	18.6%
16	1	77.44	6.18	83.66	7.4%
16	2	52.19	6.14	58.37	10.5%
16	4	46.43	6.14	52.62	11.7%
16	8	47.31	8.19	55.49	14.8%
16	16	50.32	8.19	58.53	14.0%

4.5 Split-K Graph 模式延迟 (μs)

batch	chunks=1	chunks=2	chunks=4	chunks=8	chunks=16
1	56.16	31.58	19.46	14.05	10.96
4	57.95	33.18	21.02	14.85	13.06
8	57.89	33.34	20.80	18.43	19.33
16	57.98	33.39	27.36	29.44	33.38

4.6 Layer 3 配置 Sweep 结果

batch	默认配置	默认 graph (μs)	最佳配置	最佳 graph (μs)	改善
1	chunks=16, w4, s2	11.0	chunks=16, w8, s2	10.8	1.7%
8	chunks=8, w4, s2	17.4	chunks=8, w8, s1	17.1	1.6%

Sweep 全量数据 (batch=1, graph μs):

chunks	warps	stages	graph (μs)
16	4	2	11.0
16	4	1	11.0
16	4	3	11.0
16	8	1	11.0
16	8	2	10.8
16	8	3	10.9

chunks	warps	stages	graph (μ s)
16	2	1	13.2
16	2	2	14.8
16	2	3	14.9

5. 六个讨论点

讨论点 1: Arithmetic intensity 与 Tensor Core

结论： $AI \approx 29$ FLOP/byte，远低于 B300 roofline 交叉点 ~ 225 FLOP/byte，kernel 深度 memory-bound。Tensor Core 几乎无贡献，FP8 的带宽节省被反量化开销抵消。

证据： 理论 $AI = 2g = 32$ FLOP/byte（与模拟一致）。实测数据佐证——FP8 反而比 BF16 更慢：

- batch=1 graph: FP8=10.88 μ s vs BF16=8.96 μ s → **FP8 更慢 21%**
- batch=8 graph: FP8=18.24 μ s vs BF16=12.96 μ s → **FP8 更慢 41%**
- batch=16 graph: FP8=27.23 μ s vs BF16=16.96 μ s → **FP8 更慢 61%**

如果 kernel 是 memory-bound 且 FP8 能有效利用带宽优势，FP8 应该更快。但实测 FP8 更慢，说明 FP8→BF16 反量化在关键路径上增加了计算开销，且 Tensor Core 未被有效利用。**局限：** 缺少 Nsight Compute 的 DRAM throughput 实测数据来精确量化。

讨论点 2: partial 与 merge 是否值得融合

结论： batch=1 时 merge 占 37%，理论上值得融合。但降低 chunks 至 8 即可显著降低 merge 占比至 32%，且融合需 inter-CTA 通信，工程成本远超收益。建议不融合。

证据： Split-K sweep 实测：

- batch=1, chunks=16: merge 17.52 μ s / total 46.86 μ s = **37.4%**
- batch=1, chunks=8: merge 14.88 μ s / total 46.62 μ s = **31.9%**
- batch=16, chunks=4（默认）: merge 6.14 μ s / total 52.62 μ s = **11.7%**

乐观加速上限（完全消除 merge，batch=1, chunks=16）：

$S_{\max} = 46.86 / (46.86 - 17.52) = 1.59\times$ (59% 加速) 但 batch=16 的 merge 只占 11.7%，低于 15% 阈值，不值得融合。**局限：** 未实际实现融合原型。

讨论点 3: TMA 与两级间接寻址

结论： 静态 TMA tensor map 无法表达 $topk_idx \rightarrow block_table \rightarrow physical\ page$ 的两级间接寻址。当前 Triton kernel 使用普通 global load + vector load 路径，这是合理的。

证据： 实测 split-K 数据显示，chunks=1 时 partial kernel 耗时 $\sim 76\mu$ s（batch=1 和 batch=16 均如此），且与 batch 大小几乎无关。说明延迟由 HBM 随机访问决定，而非带宽。理论分析：TMA tensor map 要求静态

base pointer + strides，不支持运行时间接寻址。动态 descriptor 创建成本 ~2-4μs，对 128B page row 可能超过搬运本身。**局限：** 未实测连续 vs 随机 page 的延迟差异。

讨论点 4：FP8 scale 处理

结论： scale 应在 K/V load 后立即应用（当前实现）。但实测表明 FP8 反量化开销显著，在 memory-bound 场景下 FP8 反而比 BF16 更慢。

证据： FP8 vs BF16 实测对比（graph 模式）：

batch	FP8 (μs)	BF16 (μs)	FP8 超出
1	10.88	8.96	+21%
8	18.24	12.96	+41%
16	27.23	16.96	+61%

FP8 需要将 1-byte FP8 转换为 2-byte BF16 并乘以 scale，这个反量化操作在 critical path 上。batch 越大，反量化总数据量随 batch 线性增长，FP8 开销越显著。**局限：** 未测试 per-token scale。

讨论点 5：Triton 基线瓶颈分析

结论： 瓶颈分为两个 regime：batch=1 时主瓶颈是 host launch overhead（占 75%），其次是 memory bandwidth 和低 occupancy。batch≥16 时主瓶颈是 partial kernel 的 HBM 访问延迟，merge 占比降至 12%。

证据： batch=1 graph 模式时间分解（推算）：

- partial kernel: ~6.5μs (59%)
- merge kernel: ~3.5μs (32%)
- graph replay: ~1.0μs (9%)
- 总计: ~10.9μs

batch=1 eager 模式时间分解：

- host launch overhead: ~33.0μs (75%)
- partial kernel: ~7.5μs (17%)
- merge kernel: ~3.5μs (8%)
- 总计: ~43.9μs

CUDA Graph 效果：

- batch=1: 43.9→10.9μs (4.0×加速)
- batch=16: 48.6→27.2μs (1.8×加速)

局限： 缺少 ncu 的 occupancy 和 DRAM throughput 数据，时间分解是基于 split-sweep 数据推算的。

讨论点 6：验收方案

结论： FP32 gathered-KV reference 作为一级参照，Triton baseline 作为二级参照。所有测试通过。

证据： B300 实测正确性：

- batch=1 BF16: max_abs=2.44e-4, cosine=0.99999535, PASS
- batch=1 FP8: max_abs=2.44e-4, cosine=0.99999529, PASS
- batch=4 BF16: max_abs=4.88e-4, cosine=0.99999541, PASS
- batch=4 FP8: max_abs=4.88e-4, cosine=0.99999547, PASS

6. Layer 3 挑战实验

6.1 Go/No-Go 决策

条件	阈值	实测	判定
merge 占比 > 15%	15%	batch=1: 37%, batch=16: 12%	⚠️ batch=1 满足
最佳 split 配置 >5% 收益	5%	见 sweep 结果	❌ 不满足
roofline 与实测间 >20% 空间	20%	launch gap 占 75%	✅ 满足

决策：GO（有条件）， 选择轻量 Triton 调参路径。

6.2 配置 Sweep 结果

batch=1（chunks=16）：

num_warps	num_stages	graph (μs)
2	1	13.2
2	2	14.8
2	3	14.9
4	1	11.0
4	2	11.0 (默认)
4	3	11.0
8	1	11.0
8	2	10.8 (最佳)
8	3	10.9

batch=8 (chunks=8) :

num_warps	num_stages	graph (μs)
2	1	21.2
2	2	21.4
2	3	21.2
4	1	17.3
4	2	17.4 (默认)
4	3	18.2
8	1	17.1 (最佳)

6.3 挑战实验结论

改善幅度仅 1.6~1.7%，远低于 5% 噪声底线，不构成稳定收益。 num_warps=8 比 4 有微弱优势 (batch=1: 10.8 vs 11.0μs)，但不超过噪声。 num_stages 几乎无影响 (差异 <0.2μs)。

7. 最终结论

在 B300、TP1、batch≤16 下，即使完全消除 merge kernel，理论加速上限也只有 1.59× (batch=1, chunks=16)；实测 Triton 调参最佳收益仅 1.7% (低于 5% 噪声底线)。主要瓶颈是 host launch overhead (eager 模式占 75%) 和 FP8 反量化开销 (比 BF16 慢 21~60%)。CUDA Graph 已将 launch overhead 消除至 ~1μs，进一步优化空间有限。考虑到专用 kernel 还需处理 paged KV 两级索引、FP8 scale、softmax reduction 和 CUDA Graph 兼容性，工程成本远超收益。建议：使用 CUDA Graph + 默认 split-K 配置，不新增 CUDA/CUTLASS 路径。

负结论证据链

证据项	数据
Triton 当前时间分解	batch=1: launch 75%, partial 17%, merge 8% (eager); graph 下 partial 59%, merge 32%
完全消除 merge 理论加速上限	batch=1, chunks=16: 1.59×; batch=16, chunks=4: 1.13×
Split-K 配置优化收益	< 1% (所有 batch)
Triton warps/stages sweep 收益	1.7% (低于 5% 噪声底线)
FP8 vs BF16	FP8 比 BF16 慢 21~60%
CUDA Graph 效果	batch=1: 43.9→10.9μs (4.0×加速), batch=16: 48.6→27.2μs (1.8×加速)
专用 kernel 需处理的复杂度	paged KV + 两级索引 + FP8 scale + softmax + reduction + TP + CUDA Graph